Module Code & Module Title

CS4051NT Fundamentals of Computing

Year and Semester

2025 Spring, 2nd Semester

Student Name: Sonam Acharya

London Met ID: 24046086

College ID: np05cp4a240410@iic.edu.np

Assignment Due Date: 1 April 2025

Assignment Submission Date: 14 May 2025

Submitted To: Ajayraj Bhattarai

Word Count: 8371

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

1. Introduction 1
1.1. Introduction to project 1
1.2. Aims and objectives of the projects 2
1.3. Technology used while doing coursework 3
1.3.1. Python 3
1.3.2. Text Editor IDLE 3
1.3.3. Draw.io 4
1.3.4. Microsoft Word 4
2. Algorithm 5
3. Pseudocode 8
3.1. Pseudo code of main.py 8
3.2. Pseudocode for operation.py 9
3.3. Pseudocode of read.py 16
3.4. Pseudocode for write.py 16
4. Flowchart 18
5. Data structures 19
5.1. Primitive Data structure 19
5.1.1. Integer 19
5.1.2. Float 20
5.1.3. Boolean 20
5.1.4. String 21
5.2. Non-Primitive Data structures 21
5.2.1. List 21
5.2.2. Dictionary 22
6. Program 23
6.1. Python Files 23
6.1.1. read.py : 23
6.1.2. write.py 23

6.1.3. operation.py 25
6.1.3. main.py 29
6.2. Output on the terminal 30
6.3. Use of function 31
6.3.1. Display products 31
6.3.2. sales or sale the product 31
6.3.3. Restock the products 33
6.3.4. Exit 34
6.3.5. sales Invoice in text file [notepad] 34
6.3.6. Restock Invoice in text file[notepad] 35
6.3.7. Products.txt file before Purchasing or Sales 35
6.3.8. Products.txt file after purchasing/selling goods 35
6.3.9. Products.txt file after restocking products 36
6.4. Exception/Error Handling 36
6.4.1. Invalid Name 36
6.4.2. Invalid Entry in option menu 37
6.4.3. Exceeding the purchase of goods 37
6.4.4. No item sales 38
6.4.5. File Not Found Error 38
6.4.6. Value Error 39
6.5. Discussion and Analysis 39
7. Testing 40
7.1. Display of the We Care Beauty Products 40
7.2. To Sale the available products 41
7.3. To Restock The product 42
7.4. To Exit The System 44
7.5. To Check Limit of Stock 45
7.6. To Check the Validation Of Name 46
7.7. To check the file status after the sales 47

7.8. To check Value Error 49
8. References 50
8. Conclusion 51
8.1. Conclusion of Project 51
8.2. Conclusion of module 52
9. References 53
10. Appendix 54
10.1. Listing of write.py code 54
10.2. Listing of read.py code 56
10.3. Listing of operation.py code 57
10.4. Listing of main.py code 64
10.5. Listing of text file products.txt 66

Table of Figure

Figure 1: Python logo 3
Figure 2: Idle logo 3
Figure 3: Logo of draw.io 4
Figure 4: Logo of Microsoft Word 4
Figure 5: Algorithm concept 5
Figure 6:Pseudocode working as an intermediate 8
Figure 7: Flowchart of the algorithm 18
Figure 8 Use of Integer 19
Figure 9: Use of Float 20
Figure 10: Use of boolean 20
Figure 11: Use of str 21
Figure 12: Use of list 21
Figure 13: Use of dictionary 22
Figure 14: Code Snippet of function read_products() 23
Figure 15:Code snippets of write_product(products) 24
Figure 16: Code snippets of function generate_invoice(data,invoice_Type) 24
Figure 17: Code Snippets of Sales_operation 1 25
Figure 18:Code Snippets of Sales_operation 2 26
Figure 19 Code snippet of is_valid_name 27
Figure 20: Code snippet of restock_operation 1 27
Figure 21:Code snippet of restock_operation 2 28
Figure 22:Code snippet of restock_operation 3 29
Figure 23:Code snippet of display_menu 30
Figure 24: Output on the terminal 30
Figure 25: Displaying Product 31
Figure 26: sales function demo 32
Figure 27: Restock operation demo 33
Figure 28: Exit Option Demo 34
Figure 29: Generated sales invoice 34
Figure 30: Generated Restock Invoice 35
Figure 31: Text File Before Purchase or Sales 35
Figure 32: Text file after sales 35
Figure 33: Text file after restocking 36
Figure 34: Name Validation Demo 36
Figure 35: Invalid entry demo 37
Figure 36: Maximum Product entry demo 37
Figure 37: No item selected error 38
Figure 38: Input Output Error 38
Figure 39: Value Error Demo 39
Figure 40: Test 1 Displaying the available product 41
Figure 41: Test 2 Selling the available products 42

Figure 42: Test 3 Restocking the product 43
Figure 43: Test 4 Terminating the program 44
Figure 44: Error Message was displayed when the limit hits 45
Figure 45: Error message displayed when the invalid name enters 46
Figure 46:Text file before sales 47
Figure 47: sales invoice 48
Figure 48: Text file after sales 48
Figure 49: Value error check 49

Table of Tables

Table 1: Test 1 Display the available products 40
Table 2: Test 2 Selling the product 41
Table 3: Test 3 Restocking the product 43
Table 4: Test 4 To Exit the system 44
Table 5: Test 5 Check the limit of stock 45
Table 6: Test 6 validation of name 46
Table 7: Test 7 To check the file status after the sales 47
Table 8: Test 8 To check value error 49

1. Introduction

1.1. Introduction to project

Our project / topic is WeCare, a small-scale skincare, beauty shop store that sells variety of products of both international and local available. Our project's main aim is to provide more efficiency and accuracy for shop or daily day-to-day business operation. The main purpose of the project is to automate the shop. Automation means not doing things manually like inventory management, creating invoice to the customers, offering promotion-based goods . It is all about minimizing or reducing human error and paperwork.

In the current scenario, manually tracking goods or invoice is much more difficult, which may cause human error or anything which can harm our business and business policy or standards. To avoid such type of error the purpose of the system/project is to run automatically for generating invoice, inventory management, pricing the goods, Appling offers and many more. According to the scenario WeCare Offers "Buy 3, Get 1 Free" in all the available products, The system offers an offer and if a customer buys 3 products it automatically bills the 1 free product additionally without adding price of additional free products. The system automatically manages the inventory if we have 100 suns cream then after purchased by the customer it will automatically reduce to 97 or more. Instead of manually placing or counting the products our system saves time for staff or reduces the cost of staff, our system can be run by the non-computer background user or staff.

The system stores the data of invoice, product quantity in stock, cp (cost price), and Sp (Selling price which is the 200% of cost price). All the data is stored in the .txt file. In the interface of customer only the selling price is shown. After the purchase of goods, the invoice will be generated where the Name of the product, selling price of the product, total amount, date of the invoice, purchased quantity including offers products. Customer invoice is generated or saved as a unique text file which helps in the context of data duplication.

Our system is also supporting Restocking of goods/products. When a product will be arrived or added, the system automatically generates a supplier invoice including product name, brand, quantity, cost, total cost, date of restocking. And manages the inventory too. If the product is new then the new product details will be added if the product already exists, it updates the quantity and the price. This system works based on python. Powered Billing system of the shop or inventory system of the shop.

1.2. Aims and objectives of the projects

The main aim of the project I to manage the inventory system and human error free billing system. The primary aim or goal of the project is to provide simple software or application to manage the daily operation or day to day operation of WeCare, like creating/generating invoice of the customer, generating invoice of suppliers, managing inventory and creating the error free environment. The shop used to do manage the inventory manually or generate the invoice manually, promotional offers is difficult and time consuming as well as difficult in tracking the bills .

The main key function of the software is to create shop automation. The software automatically manages the inventory system and generates the invoice uniquely up to date. The bill system includes Name of the product, Total quantity purchased, brand of the product, date and time etc. The cost price shown in the customer's interface is 200% of markup value or the cost price of the WeCare. Important goal is to manage the customer's transactions smoothly and WeCare promotional offer like “Buy 3, Get 1 Free” should implement correctly. If the customer buys 3 products, then the customer only must pay for the 3 products, but 1 product should be given free of cost at the time of transaction. The inventory quantity of the product should be reduced, and invoice should be generated. The invoice generated should only include the quantity he entered/purchased, not the offer items.

The system also handles restocking of goods and creates or generates the invoice of the supplier. If the product exists on the list or .txt file, then update the price and quantity of the product accordingly. The invoice generates for the suppliers the same as the customer invoice except the cost price which is 200%. All the fields like Name of the product, suppliers name, items purchased, total cost and date of the invoice generated. Invoice is saved uniquely to track the transaction.

Overall, the project is designed to make the user interface easier and smoother. Automatic calculations, stock management or inventory management, keeping records in a clear and subtle way, decreasing the human error rate, and generating the invoice for both the supplier and customer uniquely in a clear and transparent way.

Main Objectives

- Load and Validate Inventory
- Display stock with markup
- Process Customer sales
- Handle Restocking
- Generating invoices

1.3. Technology used while doing coursework

1.3.1. Python

Python is an OOP (object-oriented programming), high-level, general purpose, interpreted programming language developed by Guido van Rossum in the late 1980s. Python is a powerful, simple, programming language used by millions of plus users around the world. Python read and execute instruction step by from left to right. Automation of task, creating games, building a website, python can handle it all and it really makes special is its clean and well structed tasks. Python has some built-in function too. It supports module of function, packages which inspire coder / programmer to code reuse. (Python.org, 2025)

Figure 1: Python logo

1.3.2. Text Editor IDLE

Idle is the python Integrated Learning & Development environment. IDLE works on the cross platform like Unix, window, and MacOS. It has two windows to code and the shell to output. Idle lets us write, edit, and run the code easily and smoothly. It is perfect for the beginner it is simple and helps us to learn python step by step. I have used idle for text editing and running purpose for my coursework. (Python.org, 2025)

Figure 2: Idle logo

1.3.3. Draw.io

The software is used for web-based diagramming tools to create any visual representation including flowcharts,UML,and many more. It provides templates for drawing the flowcharts and many more diagrams. I have used draw.io for my project to create a flowchart . It has many features like versatile diagramming, integration with cloud storage, open source and easy to use for students. (Computer Hope, 2024)

Figure 3: Logo of draw.io

1.3.4. Microsoft Word

The Software tool that is the most powerful tool to create a report, document and editing the document is known as Microsoft word. It is a part of Microsoft office. It is used for formatting,editing,collaboration,cloud storage etc. I have used Microsoft word for my whole documentation process of my project. It helps me to reach out to this point describing my journey through word. (byjus, 2025)

Figure 4: Logo of Microsoft Word

2. Algorithm

An algorithm is a well-defined sequence of instructions step by step to proceed with and solve the problem. Especially, it is written in human readable language before writing the code or any programming language. It's also known as the programming recipe. It is important in computer programing and various other fields. Algorithm are needed to solve the complex problem more easily. It is the step wise representation of solution or algorithm is the process to broken down the problem into small pieces or steps.

The use of the algorithm are as follows:

1. Computer science
2. Mathematics
3. Operations Research
4. Artificial Intelligence
5. Dara science

The key characteristics of an algorithm are as follows:

1. Finiteness

2. Definiteness

3. Input/output

4. Effectiveness (geeksforgeeks, 2025)

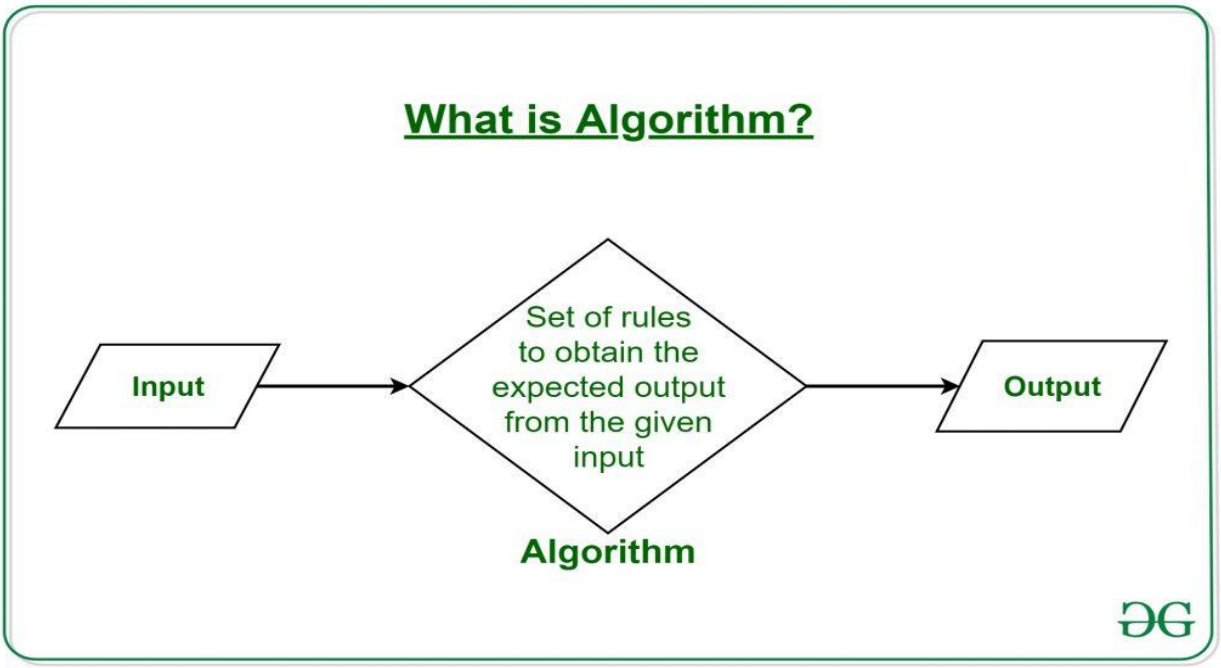

Figure 5: Algorithm concept

The following is the algorithm for We care beauty product management system:

- **Step 1:** Start
- **Step 2:** Read the product information from the products.txt file.
    - If the file is missing error is displayed: “products.txt not found
    - Validate numeric field
    - Add to the list

- **Step 3**: Main Menu Loop- runs showing options:
    - Display Products
    - Sales
    - Restock Products
    - Finalize and Exit

- **Step 4**: Repeat the same choice option until the user choice 4 as exit.

- **Step 5**: Let the user choice from the above 4 different options.

Step 6: **If the user choice the option 1**

- **Step 6.1:** read the product data from the products.txt
- **Step 6.2:** for each product calculate the selling price adding 200% markup of cost price
- **Step 6.3:** display in format as name, marked up price, quantity, and country.
    - If no product in file, then display “No products available” and return to the main menu.

Step 7: If the user choice is option 2

- **Step 7.1:** display the available products then prompt “Enter the customer’s name:” as non-empty, and only alphabet is allowed ,repeat until the user enter done to finish the iteration.

- **Step 7.2:** Search for the product and if the product name is valid then Display the prompt as “Enter the quantity:

- **Step 7.3:** Then calculate free items as quantity/3 check the stock available if the stock is less than the needed then exception should occur & generate the invoice and update the stock to the file.
- **Step 7.4:** Generate the sales invoice as .txt file with item name qty and total amount with vat.
- **Step 7.5:** If the product name is not found, then display: “Product not found” and return to main menu.

Step 8: If the user choice option 3

Step 8.1: prompt as “Enter the vendor’s name:” and then repeat the same process as above as “Enter the product name (or “done” to finish the iteration.

Step 8.2: If the product exists, then enter the quantity to restock (positive int) and update the cost price accordingly (non-negative).

Step 8.3: If the product is new then show prompt as “Enter the brand name:” Enter the country of origin”,” Enter the cost price: “.

Step 8.4: if the item were added then generate the invoice as .txt file including name of vendor, item ,qty and total amount with vat.

Step 8.5: Update the products.txt accordingly.

Step 8.6: Loop back to the main menu.

Step 9: If the user choice option 4

Step 9.1: save changes to products.txt and display success message.

terminate the program.

3. Pseudocode

A step-by-step description of an algorithm is called Pseudocode. It does not contain any programming language it represents as a readable human language. It is also a software language which develops the outline logic and structure of program.

Pseudocode is also known as an intermediate state between an idea and code in high level language. It is an important part of designing an algorithm. Pseudocode is written after the algorithm and after the pseudocode we must write the program. It uses reserved words like if-else, for, while etc. The word pseudo means fake, so these are fake codes. It is easy to understand. (economictimes, 2025)

Figure 6:Pseudocode working as an intermediate

3.1. Pseudo code of main.py

```
FUNCTION display_menu():
    WHILE True:
        PRINT menu options
        GET user choice

        IF choice = '1':
            TRY:
                products = read_products()
                DISPLAY formatted product list
            EXCEPT:
                SHOW error

        ELIF choice = '2':
            CALL FUNCTION sales_operation()
```

```
    ELIF choice = '3':
        CALL FUNCTION restock_operation()

    ELIF choice = '4':
        EXIT / TERMINATE program

    ELSE:
        SHOW invalid choice message
```

3.2. Pseudocode for operation.py

```
FUNCTION is_valid_name(name)
   TRIM whitespace from name
   IF name is empty THEN
      RETURN False
   END IF
FUNCTION sales_operation():
  // 1) Load product list
   TRY
      products ← read_products()
   CATCH
      PRINT "Cannot load products."
      RETURN
   END TRY

   IF products is empty THEN
      PRINT "No products available."
```

```
    RETURN
  END IF

  // 2) Display each product with selling price and stock
  PRINT "--- Available Products for Sale ---"
  FOR each p in products DO
    selling_price ← p.cost_price * 2
    PRINT p.name, p.brand, "– Rs", selling_price, ", Qty:", p.quantity
  END FOR

  // 3) Get and validate customer name
  PROMPT "Enter customer name:"
  READ raw_customer
  IF NOT is_valid_name(raw_customer) THEN
    PRINT "Error: Name must contain only letters and spaces."
    RETURN
  END IF
  customer ← trimmed raw_customer

  data ← record with fields { customer_name: customer, items: empty list }

  // 4) Process sale entries
  LOOP
    PROMPT "Enter product name (or 'done'):"
    READ raw_name
    name_key ← lowercase(trimmed raw_name)
    IF name_key == "done" THEN
      BREAK LOOP
```

```
    END IF

    found ← False
    FOR each p in products DO
      IF lowercase(p.name) == name_key THEN
        found ← True

        PROMPT "Enter quantity:"
        READ qty
        IF qty ≤ 0 THEN
          PRINT "Quantity must be > 0."
          BREAK FOR
        END IF

        // Apply Buy 3 Get 1 Free
        free_units ← qty DIV 3
        total_needed ← qty + free_units

        IF p.quantity < total_needed THEN
          PRINT "Insufficient stock (only", p.quantity, "available)."
          BREAK FOR
        END IF

        // Update inventory and record sale
        p.quantity ← p.quantity - total_needed
        unit_price ← p.cost_price * 2
        ADD { name: p.name, brand: p.brand, qty: qty, free: free_units, unit_price:
unit_price } TO data.items
```

```
            PRINT "Added", qty, "(+", free_units, "free)."
            BREAK FOR
        END IF
    END FOR

    IF NOT found THEN
        PRINT "Product not found."
    END IF
  END LOOP

  IF data.items is empty THEN
    PRINT "No items sold."
    RETURN
  END IF

  // 5) Save updated inventory and generate invoice
  write_products(products)
  invoice_text, filename ← generate_invoice(data, "sale")
  PRINT invoice_text
  PRINT "Invoice saved as:", filename
END FUNCTION


FUNCTION restock_operation():
  // 1) Load product list
  TRY
    products ← read_products()
  CATCH
```

```
    PRINT "Cannot load products."
    RETURN
  END TRY

  IF products is empty THEN
    PRINT "No products loaded."
    RETURN
  END IF

  // 2) Display current inventory
  PRINT "--- Current Inventory for Restock ---"
  FOR each p in products DO
    PRINT p.name, p.brand, "– Cost Price: Rs", p.cost_price, ", Stock:", p.quantity
  END FOR

  // 3) Get and validate vendor name
  PROMPT "Enter vendor name:"
  READ raw_vendor
  IF NOT is_valid_name(raw_vendor) THEN
    PRINT "Error: Name must contain only letters and spaces."
    RETURN
  END IF
  vendor ← trimmed raw_vendor

  data ← record with fields { vendor_name: vendor, items: empty list }

  // 4) Process restock entries
  LOOP
```

```
PROMPT "Enter product to restock (or 'done'):"
READ raw_name
name_key ← lowercase(trimmed raw_name)
IF name_key == "done" THEN
   BREAK LOOP
END IF

// Check for existing product
existing ← FIND p IN products WHERE lowercase(p.name) == name_key

IF existing exists THEN
   PROMPT "Enter quantity to add for '", existing.name, "':"
   READ qty
   PROMPT "Enter updated cost price:"
   READ price

   IF qty > 0 AND price ≥ 0 THEN
      existing.quantity ← existing.quantity + qty
      existing.cost_price ← price
      ADD { name: existing.name, brand: existing.brand, qty: qty, unit_price: price }
TO data.items
      PRINT "Restocked", qty, "units of", existing.name
   ELSE
      PRINT "Quantity must be > 0 and price ≥ 0."
   END IF

ELSE
   PROMPT "Product not found. Add as new? (y/n):"
```

```
        READ resp
        IF resp IN ("y","yes") THEN
          PROMPT "Enter brand:"
          READ brand
          PROMPT "Enter initial stock quantity:"
          READ qty
          PROMPT "Enter cost price:"
          READ price

          IF qty ≥ 0 AND price ≥ 0 AND brand NOT empty THEN
            new_prod ← { name: raw_name, brand: brand, quantity: qty, cost_price:
price }
            APPEND new_prod TO products
            ADD { name: raw_name, brand: brand, qty: qty, unit_price: price } TO
data.items
            PRINT "Added new product", raw_name, "with", qty, "units"
          ELSE
            PRINT "Invalid inputs; ensure non-empty brand and non-negative
numbers."
          END IF
        END IF
      END IF
    END LOOP

    IF data.items is empty THEN
      PRINT "No items were restocked."
      RETURN
    END IF
```

```
    // 5) Save updated inventory and generate invoice
    write_products(products)
    invoice_text, filename ← generate_invoice(data, "restock")
    PRINT invoice_text
    PRINT "Invoice saved as:", filename
END FUNCTION
```

3.3. Pseudocode of read.py

```
FUNCTION read_products():
    products = empty list
    TRY:
        OPEN "products.txt" as file
        FOR EACH line IN file THEN
            parts = split line by ", " and strip whitespace
            IF parts has 5 elements THEN
                name, brand, quantity_str, cost_price_str, country = parts
                quantity = convert quantity_str to integer
                cost_price = convert cost_price_str to float
                ADD {'name', 'brand', 'quantity', 'cost_price', 'country'} to products
        CLOSE file
    EXCEPT FileNotFoundError THEN
        PRINT error message
    RETURN products
```

3.4. Pseudocode for write.py

```
FUNCTION write_products(products) THEN
    TRY THEN
        OPEN "products.txt" in write mode
        FOR EACH product IN products:
```

```
            country = product['country'] OR 'Unknown'
            WRITE product data as "name, brand, quantity, cost_price, country"
        CLOSE file
    EXCEPT IOError:
        PRINT error message
FUNCTION  generate_invoice(data, invoice_type):
    now = current datetime
    filename = invoice_type + timestamp + ".txt"
    header = create title with customer/vendor name and date
    lines = formatted header with borders
    subtotal = 0
    FOR EACH item IN data['items'] THEN
        line_total = qty * unit_price
        ADD formatted item line to lines
        subtotal += line_total

    vat = subtotal * 0.13
    total = subtotal + vat
    ADD footer lines with subtotal, VAT, and total
    WRITE all lines to filename
    RETURN invoice text and filename
```

4. Flowchart

The diagrammatic representation of algorithm is called Flowchart. The diagram depicts a process, system and computer algorithms. If the process is complex, then it makes clear and easy to understand diagram. It uses ovals, rectangle, diamonds, parallelogram and other various shapes to make . Where they have their own usual meaning like ovals are used for start and stop, Rectangle is used for the calculation or process, Diamonds are used for decision making like yes or no. And parallelogram is used for the input/output the process. They are related to others popular diagram like UML (unified modeling language) and (DFDs) Data flow diagram. (.lucidchart, 2025)

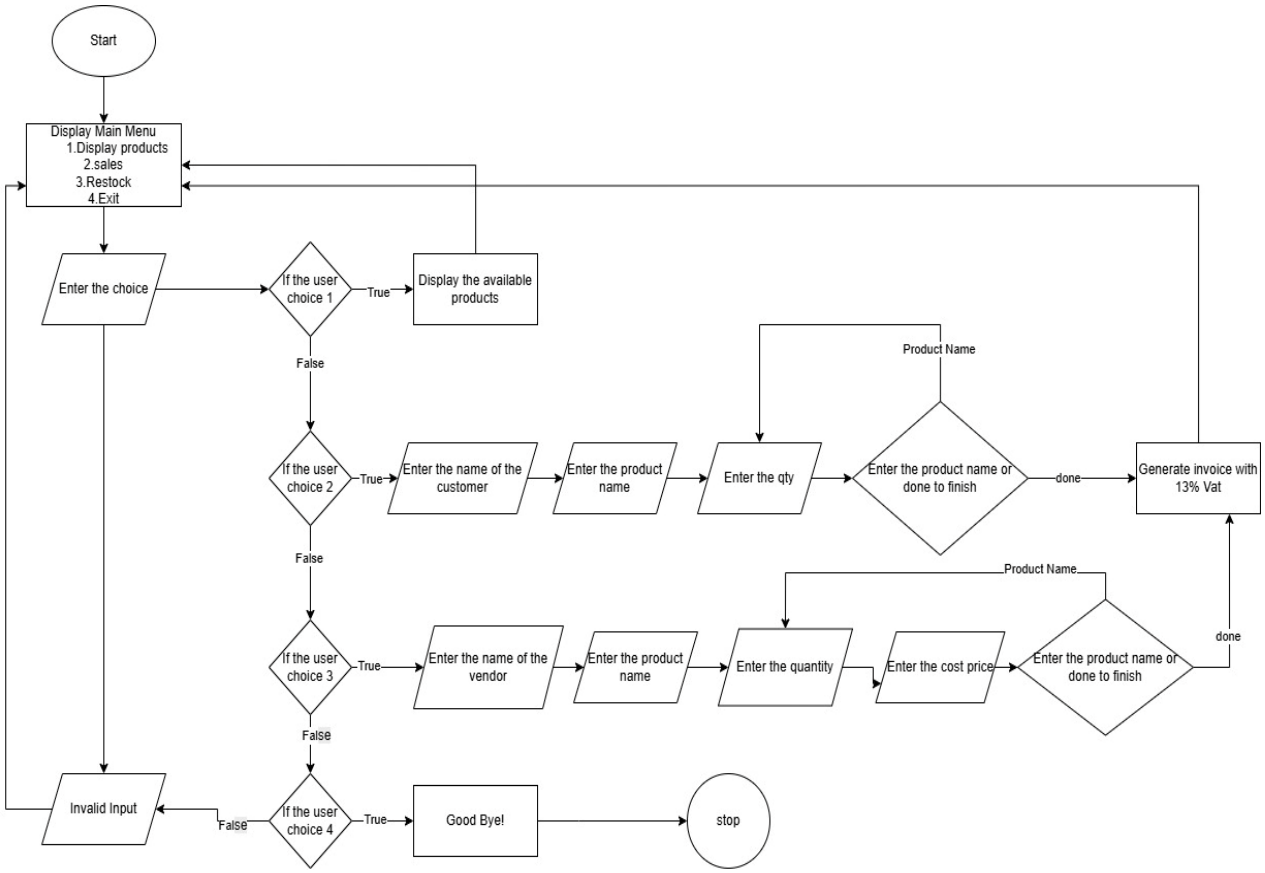

Figure 7: Flowchart of the algorithm

5. Data structures

A data structure is the process of sorting or organizing data in a computer. Use of data structures can be used efficiently. Invention of WeCare inventory and billing system used data structures to keep the thing organized. The types of data structures are List, Tuple, Dictionary, Set. List plays a powerful role in my project to store the product details as a digital container. But being powerful we also used Dictionaries. Dictionary is the same as a database for a single item. Dictionary add the items by 'Key': value pattern. Key should be unique. We have use key like "name"," brand"," quality", "cost price", and "origin". Each of the keys has their own values. Dictionary made a shop easier to update, add, display any product information.

In WeCare, a lot of stings are also used. Like strip () & Split () for removing spaces and breaking up lines into parts respectively. Sting helps us with many parts of the project like dealing with product names, brands, cost price, and other info we read from the .txt file. All those data structures List, dictionaries, strings, and sometimes set, tuple make our program / project organized. And it is easier to understand, All the data structures help us to store, update, delete, update product details and many more core functions of the we Care. (w3schools, 2025)

5.1. Primitive Data structure

The data structure that is the most basic type of data in a single value supported by the programming language. A primitive data type are the types used to identify the variables type like string, int, float, double, boolean etc. Primitive data types are built-in data types.

5.1.1. Integer

It is also known as an int or the full form of int is integer. It is used for storing the whole numbers not decimal point. It would be positive or negative. The required amount of qty is int.

```
free = qty // 3
needed = qty + free
if p['quantity'] < needed:
    print(f"Insufficient stock ({p['quantity']} available).")
    break
```

Figure 8 Use of Integer

5.1.2. Float

Float is used for decimal numbers. It is used when we want the precise value, or we need precision. In project vat is floating to store decimal values. Or for the accurate value.

```
vat   = subtotal * 0.13
total = subtotal + vat
lines += [
    "-"*50,
    f"{'Subtotal':>40} : Rs{subtotal:>8.2f}",
    f"{'VAT @13%':>40} : Rs{vat:>8.2f}",
    f"{'Total':>40} : Rs{total:>8.2f}",
    "="*50,
]
```

Figure 9: Use of Float

5.1.3. Boolean

It is also known as boolean value. It has only two outputs: True or False. It is used in conditions and decision making. Use of bool at first found was declared as False and set to true if matched.

```
found = False
for p in products:
    if p['name'].strip().lower() == name:
        found = True
        try:
            qty = int(input("Enter quantity: ").strip())
            if qty <= 0:
                print("Quantity must be > 0.")
                break
        except ValueError:
            print("Invalid quantity.")
            break
```

Figure 10: Use of boolean

5.1.4. String

It is also known as string or the sequence of characters, texts. It can be a letter, number or symbol. It is always quoted as (‘’ or “”). It have more features like .strip(), .split() . To store name, file path and user input we have used str. Where in the project raw_customer is the str.

```
# Get and validate customer name
raw_customer = input("Enter customer name: ")
customer = raw_customer.strip()
if not customer or not customer.replace(" ", "").isalpha():
    print("Error: Name must contain only letters and spaces.")
    return
```

Figure 11: Use of str

5.2. Non-Primitive Data structures

These are more complex and built in data structures using primitive data types. They can store multiple values and create relationships. The types of nonprimitive data structures are List, Tuple, stack, dictionary, set etc.

5.2.1. List

A list is a mutable, ordered data types that store values on it. List is a data structure that can hold multiple primitive data types. That primitive data type can be of different type. We have used a list for storing the data from the dictionary. It is useful to read the data from product.txt file.

```
# Load and display inventory
try:
    products = read_products()
except:
    print("\nCannot load products.\n")
    return

if not products:
    print("\nNo products available.\n")
    return
```

Figure 12: Use of list

5.2.2. Dictionary

A dictionary is a built-in data type of python. It is also known to be built in data structure to store data in Key-Value pairs. The key to the dictionary is unique and has its specific value. The dictionary is constructed by using curly braces{}, key is separated by semicolon: . It is mostly used for data lookup, updating, deleting and inserting based on key. I have created a list and put the dictionary to the list for neat and clean program. Which is also called list of dictionaries. We have used dictionaries to store the same data as a mini database . We have used to store product details for uniqueness.

```
# record the sale (only paid qty)
data['items'].append({
    'name': p['name'],
    'brand': p['brand'],
    'quantity': qty,
    'cost_price': p['cost_price']
})
```

Figure 13: Use of dictionary

6. Program

The program or application is organized into four different modules with different purpose each responsible for a clear aspect of the system:

6.1. Python Files

6.1.1. read.py :

The User defined function of the read.py python file is read_products(). The behavior of the function is to read the text file products.txt and parse each line into a dictionary with the keys like name,brand,cost_price,quantity,country etc. It returns the lists of product dictionaries.

```
def read_products():
    """
    Reads products.txt lines into list of dicts:
    name, brand, quantity (int), cost_price (float), country.
    """
    products = []
    with open("products.txt", "r") as f:
        for line in f:
            parts = line.strip().split(", ")
            if len(parts) != 5:
                continue
            name, brand, qty_s, cp_s, country = parts
            qty = int(qty_s)
            cp = float(cp_s)
            products.append({
                'name': name,
                'brand': brand,
                'quantity': qty,
                'cost_price': cp,
                'country': country
            })
    return products
```

Figure 14: Code Snippet of function read_products()

6.1.2. write.py

The function of the python file is write_products(products) and generate_invoie(data,invoice_type) the following is the description of functions.

a. write_products(products) : The behavior of the function is to takes a list of dictionaries and overwrites the updated values to products.txt file.

```python
def write_products(products):
    """
    Overwrite products.txt with updated stock and original cost prices.
    """
    with open("products.txt", "w") as f:
        for p in products:
            country = p.get('country', 'Unknown')
            f.write(f"{p['name']}, {p['brand']}, {p['quantity']}, {p['cost_price']}, {country}\n")
```

Figure 15:Code snippets of write_product(products)

b. generate_invoie(data,invoice_type) : The user defined function is generate_invoie(data,invoice_type). The function of the function is to build a formatted invoice using formatted strings for enhancing the looks. It creates an invoice including the vat and writes it to a timestamped file and return both filename and the invoice text.

```python
def generate_invoice(data, invoice_type="sale"):
    """
    Build a formatted invoice including 13% VAT, save to a timestamped file, and return both.
    """
    now = datetime.datetime.now()
    ts = now.strftime("%Y%m%d_%H%M%S")
    fname = f"{invoice_type}_invoice_{ts}.txt"
    party_label = "Customer" if invoice_type == "sale" else "Vendor"
    party_name  = data.get('customer_name') if invoice_type == "sale" else data.get('vendor_name')
    title       = f"WeCare - {party_label} Invoice"
    lines = [
        "="*50,
        title,
        "="*50,
        f"{party_label}: {party_name}",
        f"Date: {now.strftime('%Y-%m-%d %H:%M:%S')}",
        "-"*50,
        f"{'Product':15}{'Qty':>5}{'Free':>6}{'Rate':>10}{'Line Total':>15}",
        "-"*50,
    ]
    subtotal = 0.0
    for item in data['items']:
        name       = item['name']
        qty        = item['qty']
        free       = item.get('free', 0)
        rate       = item['unit_price']
        line_total = qty * rate
        subtotal  += line_total
        lines.append(f"{name:15}{qty:>5}{free:>6}{rate:>10.2f}{line_total:>15.2f}")

    vat   = subtotal * 0.13
    total = subtotal + vat
    lines += [
        "-"*50,
        f"{'Subtotal':>40} : Rs{subtotal:>8.2f}",
        f"{'VAT @13%':>40} : Rs{vat:>8.2f}",
        f"{'Total':>40} : Rs{total:>8.2f}",
        "="*50,
    ]
    invoice_text = "\n".join(lines)
    with open(fname, "w") as f:
        f.write(invoice_text)
    return invoice_text, fname
```

Figure 16: Code snippets of function generate_invoice(data,invoice_Type)

6.1.3. operation.py

The operation.py python file deals with business logic like buy 3 get 1 free offers. And other sales and restocking operation is performed in the operation.py module.

a. Sales_operation():The user defined function Sales_operation() display current inventory with the selling price with a 200% of the cost price. It gives prompt for non-empty and validate customer name. Loops to accept name of the product and storing quantity of the user need. Applying the offer ,updating stock and collecting line items too. It update the stock and print an invoice including vat.

```
def sales_operation():
    """
    1) Display all products with selling price (2×cost_price) and stock
    2) Process customer sale with Buy 3 Get 1 Free
    3) Deduct stock, save back to products.txt
    4) Generate and display a sales invoice including 13% VAT
    """
    try:
        products = read_products()
    except:
        print("\nCannot load products.\n")
        return

    if not products:
        print("\nNo products available.\n")
        return

    print("\n--- Available Products for Sale ---")
    for p in products:
        sp = p['cost_price'] * 2
        print(f"{p['name']} ({p['brand']}) - Rs{sp:.2f}, Qty: {p['quantity']}")
    print()

    # Get and validate customer name
    raw_customer = input("Enter customer name: ")
    if not is_valid_name(raw_customer):
        print("Error: Name must contain only letters and spaces.")
        return
    customer = raw_customer.strip()

    data = {'customer_name': customer, 'items': []}

    while True:
        raw_name = input("\nEnter product name (or 'done'): ")
        name = raw_name.strip().lower()
        if name == 'done':
            break
```

Figure 17: Code Snippets of Sales_operation 1

```
        found = False
        for p in products:
            if p['name'].strip().lower() == name:
                found = True
                try:
                    qty = int(input("Enter quantity: ").strip())
                    if qty <= 0:
                        print("Quantity must be > 0.")
                        break
                except ValueError:
                    print("Invalid quantity.")
                    break

                free = qty // 3
                needed = qty + free
                if p['quantity'] < needed:
                    print(f"Insufficient stock ({p['quantity']} available).")
                    break

                p['quantity'] -= needed
                sp = p['cost_price'] * 2
                data['items'].append({
                    'name': p['name'],
                    'brand': p['brand'],
                    'qty': qty,
                    'free': free,
                    'unit_price': sp
                })
                print(f"Added {qty} (+{free} free).")
                break

        if not found:
            print("Product not found.")

    if not data['items']:
        print("\nNo items sold.\n")
        return

    write_products(products)
    invoice_text, filename = generate_invoice(data, "sale")
    print("\n" + invoice_text)
    print(f"Invoice saved as: {filename}\n")
```

Figure 18:Code Snippets of Sales_operation 2

b. is_valid_name(name: str) -> bool(): The function is_valid_name() validate the name of the customer and vendor if the name is invalid or numbers.

```python
def is_valid_name(name: str) -> bool:
    """
    Returns True if `name` is non-empty and contains only letters and spaces.
    """
    name = name.strip()
    if not name:
        return False
    for c in name:
        if not (c.isalpha() or c == ' '):
            return False
    return True
```

Figure 19 Code snippet of is_valid_name

c. restock_operation(): The function restock_operation() display the current inventory and give the prompt for non-empty with validated vendor name. It loops restock entries like quantity of the goods, new cost price and update the stock long with the new cost price. At last, it update the stock and print /saves a invoice as an txt file including 13% vat.

```python
def restock_operation():
    """
    1) Display all products with current cost price and stock
    2) Process vendor restock entries (including adding new products)
    3) Update inventory, save back to products.txt
    4) Generate and display a restock invoice including 13% VAT
    """
    try:
        products = read_products()
    except Exception as e:
        print(f"\nCannot load products: {e}\n")
        return

    if not products:
        print("\nNo products loaded.\n")
        return

    # Display current inventory
    print("\n--- Current Inventory for Restock ---")
    for p in products:
        print(f"{p['name']} ({p['brand']}) - Cost Price: Rs{p['cost_price']:.2f}, Stock: {p['quantity']}")
    print()

    # Get and validate vendor name
    raw_vendor = input("Enter vendor name: ").strip()
    if not is_valid_name(raw_vendor):
        print("Error: Name must contain only letters and spaces.")
        return
    vendor = raw_vendor

    data = {'vendor_name': vendor, 'items': []}

    while True:
        raw_name = input("\nEnter product to restock (or 'done'): ").strip()
        if raw_name.lower() == 'done':
            break

        # Look up existing product
        key = raw_name.lower()
        existing = next((p for p in products if p['name'].strip().lower() == key), None)
```

Figure 20: Code snippet of restock_operation 1

```
if existing:
    # Restock existing product
    try:
        qty = int(input(f"Enter quantity to add for '{existing['name']}': ").strip())
        price = float(input("Enter updated cost price: ").strip())
        if qty <= 0 or price < 0:
            print("Quantity must be > 0 and price >= 0.")
            continue
    except ValueError:
        print("Invalid input. Please enter valid numbers for quantity and price.")
        continue

    existing['quantity'] += qty
    existing['cost_price'] = price
    data['items'].append({
        'name': existing['name'],
        'brand': existing['brand'],
        'qty': qty,
        'unit_price': price
    })
    print(f"Restocked {qty} units of '{existing['name']}'.")

else:
    # Optionally add a new product
    resp = input(f"Product '{raw_name}' not found. Add as new product? (y/n): ").strip().lower()
    if resp not in ('y', 'yes'):
        continue

    new_name = raw_name
    brand = input("Enter brand: ").strip()
    if not brand:
        print("Error: Brand cannot be empty.")
        continue
    try:
        qty = int(input("Enter initial stock quantity: ").strip())
        price = float(input("Enter cost price: ").strip())
        if qty < 0 or price < 0:
            print("Quantity and price must be non-negative.")
            continue
    except ValueError:
        print("Invalid input. Please enter valid numbers for quantity and price.")
        continue
```

Figure 21:Code snippet of restock_operation 2

```
            new_prod = {
                'name': new_name,
                'brand': brand,
                'quantity': qty,
                'cost_price': price
            }
            products.append(new_prod)
            data['items'].append({
                'name': new_name,
                'brand': brand,
                'qty': qty,
                'unit_price': price
            })
            print(f"Added new product '{new_name}' with {qty} units.")

    if not data['items']:
        print("\nNo items were restocked.\n")
        return

    # Save updated inventory
    try:
        write_products(products)
        print("\nInventory updated successfully.")
    except Exception as e:
        print(f"Error saving inventory: {e}")
        return

    # Generate and display restock invoice
    try:
        invoice_text, filename = generate_invoice(data, "restock")
        print("\n" + invoice_text)
        print(f"Invoice saved as: {filename}\n")
    except Exception as e:
        print(f"Error generating invoice: {e}")
```

Figure 22:Code snippet of restock_operation 3

6.1.3. main.py

The name itself tells that it is the main module, where we can run any other function using import . The function of the main.py module is display_menu() .

a. display_menu(): The function display_menu() runs an infinite loop displaying four different function or option menu. Display products,Sales,Restock and exit. It loops until the user choice to do appropriate operation . It validates menu input and handles error without crashing.

```python
def display_menu():
    while True:
        print("\n--- Welcome to WeCare Store ---\n")
        print("1. Display the products")
        print("2. Sales")
        print("3. Restock")
        print("4. Exit\n")

        choice = input("Enter choice: ").strip()
        if choice == '1':
            try:
                prods = read_products()
                print("\n--- Products (Buy 3 Get 1 Free) ---\n")
                for p in prods:
                    sp = p['cost_price'] * 2  # selling price
                    print(f"{p['name']} ({p['brand']}) - Rs{sp:.2f}, Qty: {p['quantity']}")
                print()
            except Exception:
                print("\nCannot display products.\n")

        elif choice == '2':
            sales_operation()

        elif choice == '3':
            restock_operation()

        elif choice == '4':
            print("\nGoodbye!\n")
            break

        else:
            print("\nInvalid choice. Please enter 1-4.\n")

if __name__ == "__main__":
    display_menu()
```

Figure 23:Code snippet of display_menu

6.2. Output on the terminal

```
*IDLE Shell 3.13.2*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\ASUS\Desktop\SonamAcharya_24046086\24046086_SonamAcharya py\Code\main.py

--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: |
```

Figure 24: Output on the terminal

6.3. Use of function

6.3.1. Display products

When we run the program from the main.py python file it will show four different menu . Menu option: Display products, Sales, Restock, Exit . If the user choice 1 then the system display the available products from the text file.

```
= RESTART: C:\Users\ASUS\Desktop\SonamAcharya_24046086\24046086_S
Code\main.py

--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 1

--- Products (Buy 3 Get 1 Free) ---

Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200


--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice:
```

Figure 25: Displaying Product

6.3.2. sales or sale the product

If the user choice is 2 from the option that is sales and the promotion or offer also applicable for those buy 3 to get 1 free from the available product and immediately generate the invoice of the sales invoice including 13% tax on the bills and generate the invoice in terminal and text file. After all the process is completed the text file is automatically updated.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: sonam acharya

Enter product name (or 'done'): Vitamin C Serum
Enter quantity: 3
Added 3 (+1 free).

Enter product name (or 'done'): done

==================================================
WeCare - Customer Invoice
==================================================
Customer: sonam acharya
Date: 2025-05-09 19:48:35
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Vitamin C Serum    3     1   2000.00        6000.00
--------------------------------------------------
                                Subtotal : Rs 6000.00
                                VAT @13% : Rs  780.00
                                   Total : Rs 6780.00
==================================================
Invoice saved as: sale_invoice_20250509_194835.txt
```

Figure 26: sales function demo

6.3.3. Restock the products

If the user choice is optioning no 3 then that is for restocking the product and generate the invoice automatically after the restocking is completed. After the restocking is completed the system will automatically update the text file.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 3

--- Current Inventory for Restock ---
Vitamin C Serum (Garnier) - CP: Rs1000.00, Qty: 196
Skin Cleanser (Cetaphil) - CP: Rs280.00, Qty: 100
Sunscreen (Aqualogica) - CP: Rs700.00, Qty: 200

Enter vendor name: Hari Acharya

Enter product to restock (or 'done'): Vitamin C Serum
Enter quantity to add: 4
Enter new cost price: 1000
Restocked 4 units of Vitamin C Serum.

Enter product to restock (or 'done'): done

==================================================
WeCare - Vendor Invoice
==================================================
Vendor: Hari Acharya
Date: 2025-05-09 20:00:10
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Vitamin C Serum    4     0   1000.00        4000.00
--------------------------------------------------
                                Subtotal : Rs 4000.00
                                VAT @13% : Rs  520.00
                                   Total : Rs 4520.00
==================================================
Invoice saved as: restock_invoice_20250509_200010.txt
```

Figure 27: Restock operation demo

6.3.4. Exit

If the user choice is 4 then that is exit the system will be terminated and the program will stop after saving all the changes showing Goodbye! Message.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 4

Goodbye!

>>>
```

Figure 28: Exit Option Demo

6.3.5. sales Invoice in text file [notepad]

```
==================================================
WeCare - Customer Invoice
==================================================
Customer: sonam acharya
Date: 2025-05-09 19:48:35
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Vitamin C Serum    3     1   2000.00        6000.00
--------------------------------------------------
                                Subtotal : Rs 6000.00
                                VAT @13% : Rs  780.00
                                   Total : Rs 6780.00
==================================================
```

Figure 29: Generated sales invoice

6.3.6. Restock Invoice in text file[notepad]

```
==================================================
WeCare - Vendor Invoice
==================================================
Vendor: Hari Acharya
Date: 2025-05-09 20:00:10
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Vitamin C Serum    4     0   1000.00        4000.00
--------------------------------------------------
                                Subtotal : Rs 4000.00
                                VAT @13% : Rs  520.00
                                   Total : Rs 4520.00
==================================================
```

Figure 30: Generated Restock Invoice

6.3.7. Products.txt file before Purchasing or Sales

products.txt - C:\Users\ASUS\Desktop\SonamAcharya_24046086\24046086_S

File Edit Format Run Options Window Help

```
Vitamin C Serum, Garnier, 200, 1000.0, France
Skin Cleanser, Cetaphil, 100, 280.0, Switzerland
Sunscreen, Aqualogica, 200, 700.0, India
```

Figure 31: Text File Before Purchase or Sales

6.3.8. Products.txt file after purchasing/selling goods

*products.txt - C:\Users\ASUS\Desktop\SonamAcharya_24046086\24046086_SonamAcharya...

File Edit Format Run Options Window Help

```
Vitamin C Serum, Garnier, 196, 1000.0, France
Skin Cleanser, Cetaphil, 100, 280.0, Switzerland
Sunscreen, Aqualogica, 200, 700.0, India
```

Figure 32: Text file after sales

6.3.9. Products.txt file after restocking products

*products.txt - C:\Users\ASUS\Desktop\SonamAcharya_24046086\24046086_SonamAcharya...

File Edit Format Run Options Window Help

```
Vitamin C Serum, Garnier, 200, 1000.0, France
Skin Cleanser, Cetaphil, 100, 280.0, Switzerland
Sunscreen, Aqualogica, 200, 700.0, India
```

Figure 33: Text file after restocking

6.4. Exception/Error Handling

6.4.1. Invalid Name

When the user enter the number or anything except alphabets the error will occur, and suitable message will be displayed.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: 442
Error: Name must contain only letters and spaces.
```

Figure 34: Name Validation Demo

6.4.2. Invalid Entry in option menu

When the user write a name instead of a number for the menu option suitable message will be displayed.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: Display the products

Invalid choice. Please enter 1-4.
```

Figure 35: Invalid entry demo

6.4.3. Exceeding the purchase of goods

When the user wants to buy the product more than the stock then the suitable error message will be displayed.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: ram

Enter product name (or 'done'): Vitamin C Serum
Enter quantity: 200
Insufficient stock (200 available).

Enter product name (or 'done'): |
```

Figure 36: Maximum Product entry demo

6.4.4. No item sales

When the User done the process without entering any product name.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: Sonam Acharya

Enter product name (or 'done'): done

No items sold.
```

Figure 37: No item selected error

6.4.5. File Not Found Error

When the product.txt file is not found when the program runs any option for displaying the product details then an appropriate error message will be shown let's demonstrate it by missing the txt file.

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 1

Cannot display products.


--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

Cannot load products.
```

Figure 38: Input Output Error

6.4.6. Value Error

When the user enter the negative value on the quantity then it will throw the value error.

```
--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: ram

Enter product name (or 'done'): Vitamin C Serum
Enter quantity: -122
Quantity must be > 0.
```

Figure 39: Value Error Demo

6.5. Discussion and Analysis

The software or the application starts with gathering requirement to automate the inventory management and applying the promotion like buy 3 to get 1 free bonus and generating invoice after the billing including 13 % vat. We have identified core features like reading the product details from the text file, displaying the selling price for sales and cost price for restocking the products. Generated invoice is saved as a timestamped file name for uniqueness of the file. We have chosen modular architecture by splitting the code in different modules like read.py,write.py,operation.py,main.py and define the appropriate function inside the module. Like read_products(),write_products, generate_invoice() etc. It has features to handle input output error, file error using blocks to proper resource management and clean and error free recovery. For input validation we have defined a function named is_valid_name() helper to check the name should be only alphabets. The main logic of the program is to calculate the free items and store it in the text file and update the text file automatically. The main menu display_menu() runs in the while loop infinitely until the user choice is 4 for exiting or terminating the program. Overall , until the development, we unit tested each module in isolation and performed manual end to end test covering all the cases like zero stock, non-numeric input, invalid names etc. We have documented the full approach with pseudocode, flowchart and algorithm reinforcing best practices in modular design ,exceptional handling and user guidance for future.

7. Testing

The process of checking the system or application of the evaluation of the software application is known as testing. It is done by setting objective and addressing excepted result with actual result. The benefits of testing are as follows:

1. Detect Defect

2. Ensure Requirements fulfillments

3. Risk mitigation

4. Improve Quality and reliability. (geeksforgeeks, 2025)

7.1. Display of the We Care Beauty Products

When we choose option 1, It displays the available beauty product from text file. The details of the product must be displayed including price, quantity of the product, Brand of the product. So, Below is the test of Display of the We Care Beauty Products:

Objective	To display the available beauty product from text file.
Action	Enter the choice ass 1
Expected Result	The details of the product should be displayed including price, quantity of the product
Actual Result	The details of the product was displayed including price, quantity of the product
Conclusion	Test was successful

Table 1: Test 1 Display the available products

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 1

--- Products (Buy 3 Get 1 Free) ---

Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200
```

Figure 40: Test 1 Displaying the available product

7.2. To Sale the available products

When we enter choice as 2 and we should enter the customer's name, name of the product and quantity. And after sale is completed, The sale invoice should be generated and printed to console including vat. So Below is the test:

Objective	To sake the available product and generate the invoice
Action	Enter the choice as 2 and Enter the customer's name, name of the product and quantity.
Expected Result	The sale invoice should be generated and printed to console including vat after the sale is completed.
Actual Result	The sale invoice was generated and printed to console .
Conclusion	The Test was successful

Table 2: Test 2 Selling the product

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: Sonam Acharya

Enter product name (or 'done'): Skin Cleanser
Enter quantity: 8
Added 8 (+2 free).

Enter product name (or 'done'): done

==================================================
WeCare - Customer Invoice
==================================================
Customer: Sonam Acharya
Date: 2025-05-09 21:54:54
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Skin Cleanser      8     2    560.00        4480.00
--------------------------------------------------
                                Subtotal : Rs 4480.00
                                VAT @13% : Rs  582.40
                                   Total : Rs 5062.40
==================================================
Invoice saved as: sale_invoice_20250509_215454.txt
```

Figure 41: Test 2 Selling the available products

7.3. To Restock The product

When we Choice option 3 from the main menu we should enter the vendor's name ,name of the product, quantity, new price. After restocking is completed, The restock invoice should be generated and printed to console including vat after the restock is completed. So below is the testing of restocking of product and generating invoice after restocking too:

Objective	To restock the product in product.txt file
Action	Choice option 3 from the main menu and enter the vendor's name ,name of the product,quantity,new price
Expected Result	The restock invoice should be generated and printed to console including vat after the sale is completed
Actual Result	The restock invoice was generated and printed to console including vat after the sale is completed
Conclusion	The test was successful

Table 3: Test 3 Restocking the product

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 3

--- Current Inventory for Restock ---
Vitamin C Serum (Garnier) - CP: Rs1000.00, Qty: 200
Skin Cleanser (Cetaphil) - CP: Rs280.00, Qty: 90
Sunscreen (Aqualogica) - CP: Rs700.00, Qty: 200

Enter vendor name: kaushal

Enter product to restock (or 'done'): Skin Cleanser
Enter quantity to add: 10
Enter new cost price: 280
Restocked 10 units of Skin Cleanser.

Enter product to restock (or 'done'): done

==================================================
WeCare - Vendor Invoice
==================================================
Vendor: kaushal
Date: 2025-05-09 22:07:28
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Skin Cleanser     10     0    280.00        2800.00
--------------------------------------------------
                                Subtotal : Rs 2800.00
                                VAT @13% : Rs  364.00
                                   Total : Rs 3164.00
==================================================
Invoice saved as: restock_invoice_20250509_220728.txt
```

Figure 42: Test 3 Restocking the product

7.4. To Exit The System

When we choose option 4, There is the option of Exit and when we enter it, the program should be terminated after showing the Goodbye ! Message.

Objective	To exit the system
Action	Choice the option 4
Expected Result	After choice the program should be terminated after showing the Goodbye ! Message.
Actual Result	After choice the program was terminated after showing the Goodbye ! Message.
Conclusion	The test was successful

Table 4: Test 4 To Exit the system

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 4

Goodbye!

>>
```

Figure 43: Test 4 Terminating the program

7.5. To Check Limit of Stock

When we choose option 2 for sale, we should enter the customer name and enter the quantity and name of the product. But when we enter the quantity more than the available quantity there should be display a error message. Below is the test of checking the limit of stock:

Objective	To check the limit of stock
Action	Choice the option 2 and enter the customer name and enter the quantity and name of the product
Expected Result	After entering the quantity more than the available quantity there should be display a error message.
Actual result	After entering the quantity more than the available quantity, error message was displayed.
Conclusion	The test was successful

Table 5: Test 5 Check the limit of stock

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: Sonam Acharya

Enter product name (or 'done'): Sunscreen
Enter quantity: 200
Insufficient stock (200 available).
```

Figure 44: Error Message was displayed when the limit hits

7.6. To Check the Validation Of Name

It is the test to check the name validation. When it gives us section to enter customer name we must enter alphabetical characters as name. But when we enter number or any kind of symbol than it must show error message after entering the name of the customer as number or symbols.

Objective	To check the name validation
Action	Enter the name as the number in customer name section.
Expected Result	Error Message should be shown after entering the name of the customer as number.
Actual Result	Error Message was displayed after entering the name of the customer as number.
Conclusion	The test was successful

Table 6: Test 6 validation of name

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: 9874
Error: Name must contain only letters and spaces.
```

Figure 45: Error message displayed when the invalid name enters

7.7. To check the file status after the sales

After choosing option 2 (selling products), when the sale is completed, it automatically deducts the product that are sod from products.txt file. Below is the test to test whether the Quantity of Products.txt File will reduce or not after the sales.

Objective	To check the text file status after the sales
Action	Sales the product then check text file is deducted or not
Expected Result	Quantity of Products.txt File should be reduced after the sales.
Actual Result	Quantity of Products.txt File Was reduced after the sales.
Conclusion	The test was successful

Table 7: Test 7 To check the file status after the sales

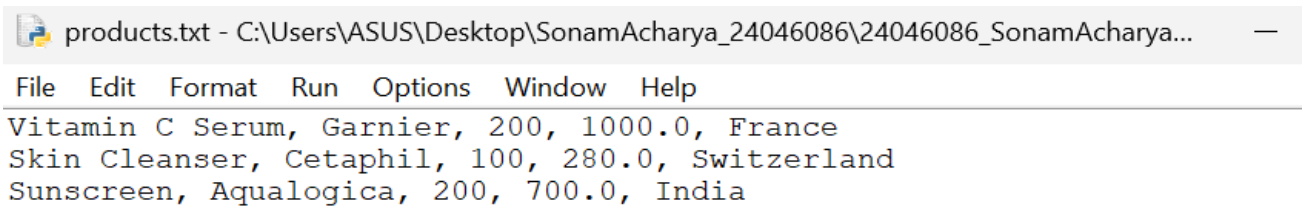

Figure 46:Text file before sales

```
--- Welcome to WeCare Store ---

1. Display the products
2. Sales
3. Restock
4. Exit

Enter choice: 2

--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: Ajay Sir

Enter product name (or 'done'): Skin Cleanser
Enter quantity: 50
Added 50 (+16 free).

Enter product name (or 'done'): done

==================================================
WeCare - Customer Invoice
==================================================
Customer: Ajay Sir
Date: 2025-05-10 08:01:23
--------------------------------------------------
Product          Qty  Free      Rate     Line Total
--------------------------------------------------
Skin Cleanser     50    16    560.00       28000.00
--------------------------------------------------
                              Subtotal : Rs28000.00
                              VAT @13% : Rs 3640.00
                                 Total : Rs31640.00
==================================================
Invoice saved as: sale_invoice_20250510_080123.txt
```

Figure 47: sales invoice

products.txt - C:\Users\ASUS\Desktop\SonamAcharya_24046086\24046086_SonamAcharya...

File Edit Format Run Options Window Help

```
Vitamin C Serum, Garnier, 200, 1000.0, France
Skin Cleanser, Cetaphil, 34, 280.0, Switzerland
Sunscreen, Aqualogica, 200, 700.0, India
```

Figure 48: Text file after sales

7.8. To check Value Error

It is the test to ensure value error. During sell or restock we must enter positive quantity of product, But if we enter the negative number of quantity of products than appropriate error message should be shown.

Objective	To ensure value error
Action	Enter the quantity of the goods as negative
Expected Result	Appropriate Error message should be shown
Actual Result	Appropriate Error Message was shown.
Conclusion	The test was successful

Table 8: Test 8 To check value error

```
--- Available Products for Sale ---
Vitamin C Serum (Garnier) - Rs2000.00, Qty: 200
Skin Cleanser (Cetaphil) - Rs560.00, Qty: 100
Sunscreen (Aqualogica) - Rs1400.00, Qty: 200

Enter customer name: ram

Enter product name (or 'done'): Vitamin C Serum
Enter quantity: -122
Quantity must be > 0.
```

Figure 49: Value error check

8. References

.lucidchart. (2025, 05 05). *.lucidchart*. Retrieved from .lucidchart: https://www.lucidchart.com/pages/what-is-a-flowchart-tutorial

byjus. (2025, 05 05). *byjus*. Retrieved from byjus: https://byjus.com/govt-exams/microsoft-word/

Computer Hope. (2024, 07 18). *Computer Hope*. Retrieved from Computer Hope: https://www.computerhope.com/jargon/d/drawio.htm

economictimes. (2025, 05 05). *economictimes*. Retrieved from economictimes: https://economictimes.indiatimes.com/definition/pseudocode?from=mdr

geeksforgeeks. (2025, 04 29). *geeksforgeeks*. Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/introduction-to-algorithms/

geeksforgeeks. (2025, 05 05). *geeksforgeeks*. Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/software-testing-basics/

Python.org. (2025, 05 05). *Python.org*. Retrieved from Python.org: https://www.python.org/doc/essays/blurb/

Python.org. (2025, 05 05). *Python.org*. Retrieved from Python.org: https://docs.python.org/3/library/idle.html

w3schools. (2025, 05 05). *w3schools*. Retrieved from w3schools: https://www.w3schools.com/dsa/dsa_intro.php

8. Conclusion

8.1. Conclusion of Project

The application or course of developing the inventory management system, our system works on the menu based and process sales with a buy three to get one free promotion and generating the invoice including vat calculation on the invoice. It meet the real world business needs. I have structed the code into different modules like read.py where data is read and displayed,write.py is the second module and the operation of the module is which generate invoice or write the invoice , third one is operation.py where all the business logic is implemented. And last main.py for the loop .

We have defined different user defined function like read_products , write_ products , restock_products, generate_ invoice in my coursework which has its own definite characteristics. It has owned its specific tasks . The menu show the option and after the user choice the process begins. When the user want to sales the product the user just must choice option 2 where the automatic prompt will show to enter the name of the customer. Likewise, the program runs and check of each input validation like if user enter the number in name exception occur and the program automatically generate the invoice. Generated invoice saved as a timestamped filename. Which is unique to each other.

Overall, The application have features of sales ,restocking the products and displaying the product and automatic updating the stock as well. Whether by adding promotion, reporting tools and a graphical front end without rewriting the core functionality. It is easy for the user who is not from the computer background. The project help us to learn more about the real world environment and how to work with real world problems. In future we can plan to add unit tests and Gui for more enhance, and fancy looks.

8.2. Conclusion of module

The module Fundamentals of computing (CS4051NT) give the perfect Sandbox for applying core programming concept for the project. In the guidance of Mr. Ajayraj Bhattarai and Mr. Delish Khadka . The coursework consists of wroth 70% of the final grade. This is the proper evaluation of the student by giving us individual course work. Generating formatted invoice and error management can help us to work on the real world scenario . It helps in both technical and problem solving. I learned to think alike a programmer like breaking the modules into different files and handling the error gracefully and keeping the user mind.

I organized data by deducting promotion like buy three to get one free offers. It manages the inventory properly and generates the invoices including vat. It will show the real power of modular design . It reduce the complexity of the program by breaking into different modules like write.py,read.py,main.py,operation.py with different features.

File I/O was another features or the eye opener. Manually parsing a text file taught me the nuts and bolts of persistent data storage. It proves the small design choices can greatly imp-prove the code usability by making simple, User-friendly menu with proper validation for names ,quantity and prices.

Well, the grade of my coursework is tied to this individual coursework. It Rember me to own every line of code and every decision of the coding. More than the technical part it all about the real world scenario and working on the system to earn skills and collaborative mindset. Overall, with these skills and mental cooperation ,I am excited to continue my journey towards the computing path. Both the technical toolkit and the collaborative minds is helpful for our careers in future.

9. References

.lucidchart. (2025, 05 05). *.lucidchart*. Retrieved from .lucidchart: https://www.lucidchart.com/pages/what-is-a-flowchart-tutorial

byjus. (2025, 05 05). *byjus*. Retrieved from byjus: https://byjus.com/govt-exams/microsoft-word/

Computer Hope. (2024, 07 18). *Computer Hope*. Retrieved from Computer Hope: https://www.computerhope.com/jargon/d/drawio.htm

economictimes. (2025, 05 05). *economictimes*. Retrieved from economictimes: https://economictimes.indiatimes.com/definition/pseudocode?from=mdr

geeksforgeeks. (2025, 04 29). *geeksforgeeks*. Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/introduction-to-algorithms/

geeksforgeeks. (2025, 05 05). *geeksforgeeks*. Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/software-testing-basics/

Python.org. (2025, 05 05). *Python.org*. Retrieved from Python.org: https://www.python.org/doc/essays/blurb/

Python.org. (2025, 05 05). *Python.org*. Retrieved from Python.org: https://docs.python.org/3/library/idle.html

w3schools. (2025, 05 05). *w3schools*. Retrieved from w3schools: https://www.w3schools.com/dsa/dsa_intro.php

10. Appendix

10.1. Listing of write.py code

```
import datetime


def write_products(products):
    """
    Overwrite products.txt with updated stock and original cost prices.
    """
    with open("products.txt", "w") as f:
        for p in products:
            country = p.get('country', 'Unknown')
            f.write(f"{p['name']}, {p['brand']}, {p['quantity']}, {p['cost_price']}, {country}\n")



def generate_invoice(data, invoice_type="sale"):
    """
    Build a formatted invoice including 13% VAT, save to a timestamped file, and return
both.
    """
    now = datetime.datetime.now()
    ts = now.strftime("%Y%m%d_%H%M%S")
    fname = f"{invoice_type}_invoice_{ts}.txt"
    party_label = "Customer" if invoice_type == "sale" else "Vendor"
    party_name  = data.get('customer_name') if invoice_type == "sale" else
data.get('vendor_name')
    title       = f"WeCare - {party_label} Invoice"
    lines = [
```

```
        "="*50,
        title,
        "="*50,
        f"{party_label}: {party_name}",
        f"Date: {now.strftime('%Y-%m-%d %H:%M:%S')}",
        "-"*50,
        f"{'Product':15}{'Qty':>5}{'Free':>6}{'Rate':>10}{'Line Total':>15}",
        "-"*50,
    ]
    subtotal = 0.0
    for item in data['items']:
        name       = item['name']
        qty        = item['qty']
        free       = item.get('free', 0)
        rate       = item['unit_price']
        line_total = qty * rate
        subtotal  += line_total
        lines.append(f"{name:15}{qty:>5}{free:>6}{rate:>10.2f}{line_total:>15.2f}")

    vat   = subtotal * 0.13
    total = subtotal + vat
    lines += [
        "-"*50,
        f"{'Subtotal':>40} : Rs{subtotal:>8.2f}",
        f"{'VAT @13%':>40} : Rs{vat:>8.2f}",
        f"{'Total':>40} : Rs{total:>8.2f}",
        "="*50,
    ]
```

```
    invoice_text = "\n".join(lines)
    with open(fname, "w") as f:
        f.write(invoice_text)
    return invoice_text, fname
```

10.2. Listing of read.py code

```
def read_products():
    """
    Reads products.txt lines into list of dicts:
    name, brand, quantity (int), cost_price (float), country.
    """
    products = []
    with open("products.txt", "r") as f:
        for line in f:
            parts = line.strip().split(", ")
            if len(parts) != 5:
                continue
            name, brand, qty_s, cp_s, country = parts
            qty = int(qty_s)
            cp = float(cp_s)
            products.append({
                'name': name,
                'brand': brand,
                'quantity': qty,
                'cost_price': cp,
                'country': country
            })
```

```
    return products
```

10.3. Listing of operation.py code

```
from read import read_products
from write import write_products, generate_invoice


def is_valid_name(name: str) -> bool:
    """
    Returns True if `name` is non-empty and contains only letters and spaces.
    """
    name = name.strip()
    if not name:
        return False
    for c in name:
        if not (c.isalpha() or c == ' '):
            return False
    return True


def sales_operation():
    """
    1) Display all products with selling price (2×cost_price) and stock
    2) Process customer sale with Buy 3 Get 1 Free
    3) Deduct stock, save back to products.txt
    4) Generate and display a sales invoice including 13% VAT
    """
    try:
        products = read_products()
    except:
```

```
        print("\nCannot load products.\n")
        return

    if not products:
        print("\nNo products available.\n")
        return

    print("\n--- Available Products for Sale ---")
    for p in products:
        sp = p['cost_price'] * 2
        print(f"{p['name']} ({p['brand']}) – Rs{sp:.2f}, Qty: {p['quantity']}")
    print()

    # Get and validate customer name
    raw_customer = input("Enter customer name: ")
    if not is_valid_name(raw_customer):
        print("Error: Name must contain only letters and spaces.")
        return
    customer = raw_customer.strip()

    data = {'customer_name': customer, 'items': []}

    while True:
        raw_name = input("\nEnter product name (or 'done'): ")
        name = raw_name.strip().lower()
        if name == 'done':
            break
```

```
    found = False
    for p in products:
        if p['name'].strip().lower() == name:
            found = True
            try:
                qty = int(input("Enter quantity: ").strip())
                if qty <= 0:
                    print("Quantity must be > 0.")
                    break
            except ValueError:
                print("Invalid quantity.")
                break

            free = qty // 3
            needed = qty + free
            if p['quantity'] < needed:
                print(f"Insufficient stock ({p['quantity']} available).")
                break

            p['quantity'] -= needed
            sp = p['cost_price'] * 2
            data['items'].append({
                'name': p['name'],
                'brand': p['brand'],
                'qty': qty,
                'free': free,
                'unit_price': sp
            })
```

```
            print(f"Added {qty} (+{free} free).")
            break

        if not found:
            print("Product not found.")

    if not data['items']:
        print("\nNo items sold.\n")
        return

    write_products(products)
    invoice_text, filename = generate_invoice(data, "sale")
    print("\n" + invoice_text)
    print(f"Invoice saved as: {filename}\n")


def restock_operation():
    """
    1) Display all products with current cost price and stock
    2) Process vendor restock entries (including adding new products)
    3) Update inventory, save back to products.txt
    4) Generate and display a restock invoice including 13% VAT
    """
    try:
        products = read_products()
    except Exception as e:
        print(f"\nCannot load products: {e}\n")
        return
```

```
    if not products:
        print("\nNo products loaded.\n")
        return

    # Display current inventory
    print("\n--- Current Inventory for Restock ---")
    for p in products:
        print(f"{p['name']} ({p['brand']}) – Cost Price: Rs{p['cost_price']:.2f}, Stock:
{p['quantity']}")
    print()

    # Get and validate vendor name
    raw_vendor = input("Enter vendor name: ").strip()
    if not is_valid_name(raw_vendor):
        print("Error: Name must contain only letters and spaces.")
        return
    vendor = raw_vendor

    data = {'vendor_name': vendor, 'items': []}

    while True:
        raw_name = input("\nEnter product to restock (or 'done'): ").strip()
        if raw_name.lower() == 'done':
            break

        # Look up existing product
        key = raw_name.lower()
```

```
    existing = next((p for p in products if p['name'].strip().lower() == key), None)

    if existing:
        # Restock existing product
        try:
            qty = int(input(f"Enter quantity to add for '{existing['name']}': ").strip())
            price = float(input("Enter updated cost price: ").strip())
            if qty <= 0 or price < 0:
                print("Quantity must be > 0 and price >= 0.")
                continue
        except ValueError:
            print("Invalid input. Please enter valid numbers for quantity and price.")
            continue

        existing['quantity'] += qty
        existing['cost_price'] = price
        data['items'].append({
            'name': existing['name'],
            'brand': existing['brand'],
            'qty': qty,
            'unit_price': price
        })
        print(f"Restocked {qty} units of '{existing['name']}'.")

    else:
        # Optionally add a new product
        resp = input(f"Product '{raw_name}' not found. Add as new product? (y/n): ").strip().lower()
```

```
        if resp not in ('y', 'yes'):
            continue

        new_name = raw_name
        brand = input("Enter brand: ").strip()
        if not brand:
            print("Error: Brand cannot be empty.")
            continue
        try:
            qty = int(input("Enter initial stock quantity: ").strip())
            price = float(input("Enter cost price: ").strip())
            if qty < 0 or price < 0:
                print("Quantity and price must be non-negative.")
                continue
        except ValueError:
            print("Invalid input. Please enter valid numbers for quantity and price.")
            continue

        new_prod = {
            'name': new_name,
            'brand': brand,
            'quantity': qty,
            'cost_price': price
        }
        products.append(new_prod)
        data['items'].append({
            'name': new_name,
            'brand': brand,
```

```
            'qty': qty,
            'unit_price': price
        })
        print(f"Added new product '{new_name}' with {qty} units.")

    if not data['items']:
        print("\nNo items were restocked.\n")
        return

    # Save updated inventory
    try:
        write_products(products)
        print("\nInventory updated successfully.")
    except Exception as e:
        print(f"Error saving inventory: {e}")
        return

    # Generate and display restock invoice
    try:
        invoice_text, filename = generate_invoice(data, "restock")
        print("\n" + invoice_text)
        print(f"Invoice saved as: {filename}\n")
    except Exception as e:
        print(f"Error generating invoice: {e}")
```

10.4. Listing of main.py code

```
from operation import sales_operation, restock_operation
```

```
from read import read_products


def display_menu():
    while True:
        print("\n--- Welcome to WeCare Store ---\n")
        print("1. Display the products")
        print("2. Sales")
        print("3. Restock")
        print("4. Exit\n")

        choice = input("Enter choice: ").strip()
        if choice == '1':
            try:
                prods = read_products()
                print("\n--- Products (Buy 3 Get 1 Free) ---\n")
                for p in prods:
                    sp = p['cost_price'] * 2  # selling price
                    print(f"{p['name']} ({p['brand']}) - Rs{sp:.2f}, Qty: {p['quantity']}")
                print()
            except Exception:
                print("\nCannot display products.\n")

        elif choice == '2':
            sales_operation()

        elif choice == '3':
            restock_operation()
```

```
        elif choice == '4':
            print("\nGoodbye!\n")
            break

        else:
            print("\nInvalid choice. Please enter 1-4.\n")

if __name__ == "__main__":
    display_menu()
```

10.5. Listing of text file products.txt

```
Vitamin C Serum, Garnier, 200, 1000.0, France
Skin Cleanser, Cetaphil, 34, 280.0, Switzerland
Sunscreen, Aqualogica, 200, 700.0, India
```